Lab 7: Fuzz Testing – Pizza Order

Objective: use Hypothesis to uncover malformed-input edge cases, fix the application, and verify the resulting behavior.

1. Initial bug found

The original implementation crashed when an unknown topping was supplied because it indexed into the dictionary without checking whether the key existed.

Before Fix: Crash on invalid input

```
>>> python3 pizza_order.py
Enter toppings as CSV (e.g. cheese,olives): cheese,unknown
Traceback (most recent call last):
  File "pizza_order.py", line 7, in toppings_price
    total += TOPPINGS[t.strip().lower()]
KeyError: 'unknown'
```

2. Safety fix applied

The application now validates input types, ignores unknown toppings, safely handles unknown sizes, and prevents divide-by-zero or malformed bill strings.

After Fix: Valid input output

```
>>> python3 pizza_order.py
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): cheese,olives
Toppings Price: 70

==== Running pizza_inches ====
Enter pizza size (S/M/L): M
Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): 900/3
Per Person: 300.0
```

3. Verification

The Hypothesis-based tests pass after the fix and confirm the functions no longer crash on fuzzed inputs.

Pytest after fix

```
============================= test session starts ==============================
collected 5 items

.....                                                                    [100%

5 passed in 1.97s
```

Result: 5 tests passed in 1.97s.